Funciones más importantes de FonoKids, con su código

Las líneas corresponden al código actual del proyecto.

Preparar los datos

1 process_clip

`scripts/extraer_landmarks_npy.py:315-351`

De un video saca el .npy con los landmarks de labios.

```
315  def process_clip(landmarker, video_path, start_timestamp_ms):
316      cap = cv2.VideoCapture(video_path)
317      fps = cap.get(cv2.CAP_PROP_FPS) or 25
318      frame_ms = max(int(1000 / fps), 1)
319
320      positions = []
321      detected, total = 0, 0
322      last_valid_vector = None
323      timestamp_ms = start_timestamp_ms
324
325      while True:
326          ok, frame = cap.read()
327          if not ok:
328              break
329          total += 1
330
331          landmarks_px, timestamp_ms = detect_frame_landmarks(landmarker, frame, timestamp_ms, frame_ms)
332          vector = normalize_lip_landmarks(landmarks_px) if landmarks_px is not None else None
333
334          if vector is not None and is_outlier(vector, last_valid_vector):
335              vector = None
336
337          if vector is not None:
338              detected += 1
339              last_valid_vector = vector
340          positions.append(vector)
341
342      cap.release()
343      next_timestamp_ms = timestamp_ms + frame_ms * 5
344
345      filled = fill_missing_frames(positions)
346      if filled is None:
347          return None, detected, total, next_timestamp_ms
348
349      smoothed = smooth_positions(filled)
350      sequence = add_dynamics(smoothed)
351      return sequence, detected, total, next_timestamp_ms
```

2 normalize_lip_landmarks

scripts/extraer_landmarks_npy.py:230-249

Centra, rota y escala los 20 puntos de la boca.

```python
def normalize_lip_landmarks(landmarks_px):
    right_corner = landmarks_px[MOUTH_RIGHT_CORNER]
    left_corner = landmarks_px[MOUTH_LEFT_CORNER]

    mouth_center = (right_corner + left_corner) / 2.0
    mouth_vector = left_corner - right_corner
    scale = np.linalg.norm(mouth_vector)
    if scale < 1e-3:
        return None
    angle = math.atan2(mouth_vector[1], mouth_vector[0])

    cos_a, sin_a = math.cos(-angle), math.sin(-angle)
    rotation = np.array([[cos_a, -sin_a], [sin_a, cos_a]], dtype=np.float64)

    lip_pts = landmarks_px[LIP_INDICES].astype(np.float64)
    translated = lip_pts - mouth_center
    rotated = translated @ rotation.T
    normalized = rotated / (scale + SCALE_EPSILON)

    return normalized.astype(np.float32).flatten()
```

3 add_dynamics

scripts/extraer_landmarks_npy.py:305-312

Suma velocidad y aceleración, de 40 a 120 números por frame.

```
305  def add_dynamics(positions):
306      velocity = np.zeros_like(positions)
307      velocity[1:] = positions[1:] - positions[:-1]
308
309      acceleration = np.zeros_like(positions)
310      acceleration[1:] = velocity[1:] - velocity[:-1]
311
312      return np.concatenate([positions, velocity, acceleration], axis=1).astype(np.float32)
```

4 extract_embedding

scripts/audio_pronunciation.py:167-173

Del audio saca el vector de 1024 números de wav2vec2.

```
167  def extract_embedding(wav_path, model, feature_extractor, device):
168      waveform = load_audio_16k(wav_path)
169      inputs = feature_extractor(waveform.numpy(), sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")
170      with torch.inference_mode():
171          hidden = model.wav2vec2(inputs.input_values.to(device)).last_hidden_state
172          pooled = hidden.mean(dim=1).squeeze(0)
173      return pooled.cpu().numpy()
```

Los modelos

5 LipReadingConformer

scripts/train_landmarks_transformer.py:179-214

Tu modelo de video.

```
179  class LipReadingConformer(nn.Module):
180      def __init__(self, num_classes, input_dim, hidden_dim=128,
181                   n_layers=2, n_heads=8, dim_feedforward=256,
182                   conv_kernel_size=15, tcn_blocks=3, tcn_kernel_size=3, dropout=0.3):
183          super().__init__()
184          self.input_proj = nn.Linear(input_dim, hidden_dim)
185
186          self.tcn = TCNFeatureExtractor(
187              hidden_dim, n_blocks=tcn_blocks, kernel_size=tcn_kernel_size, dropout=dropout
188          )
189
190          self.pos_encoder = PositionalEncoding(hidden_dim)
191
192          self.conformer_encoder = ConformerEncoder(
193              d_model=hidden_dim, n_layers=n_layers, n_heads=n_heads,
194              dim_feedforward=dim_feedforward, conv_kernel_size=conv_kernel_size,
195              dropout=dropout,
196          )
197
198          self.classifier = nn.Sequential(
199              nn.Linear(hidden_dim, 64),
200              nn.ReLU(),
201              nn.Dropout(dropout),
202              nn.Linear(64, num_classes),
203          )
204
205      def forward_features(self, x, src_key_padding_mask=None):
206          x = self.input_proj(x)
207          x = self.tcn(x, key_padding_mask=src_key_padding_mask)
208          x = self.pos_encoder(x)
209          x = self.conformer_encoder(x, src_key_padding_mask=src_key_padding_mask)
210          return masked_mean_pool(x, src_key_padding_mask)
211
212      def forward(self, x, src_key_padding_mask=None):
213          pooled = self.forward_features(x, src_key_padding_mask=src_key_padding_mask)
214          return self.classifier(pooled)
```

6 AudioClassifierHead

scripts/train_audio_classifier.py:95-107

Tu modelo de audio.

```
 95  class AudioClassifierHead(nn.Module):
 96
 97      def __init__(self, in_dim, hidden_dim, num_classes, dropout):
 98          super().__init__()
 99          self.net = nn.Sequential(
100              nn.Linear(in_dim, hidden_dim),
101              nn.ReLU(),
102              nn.Dropout(dropout),
103              nn.Linear(hidden_dim, num_classes),
104          )
105
106      def forward(self, x):
107          return self.net(x)
```

7 train

scripts/train_landmarks_transformer.py:407-489

Entrena el modelo de video.

```python
def train(epochs=50, batch_size=32, max_frames=60, lr=1e-3, weight_decay=1e-3,
          val_ratio=0.2, early_stopping_patience=15, warmup_epochs=5,
          label_smoothing=0.1):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Entrenando en: {device}")

    dataset, train_loader, val_loader = build_dataloaders(
        data_dir=RUTA_DATASET, max_frames=max_frames, batch_size=batch_size,
        val_ratio=val_ratio, seed=SEED, augment_train=True,
    )
    num_classes = len(dataset.class_names)
    input_dim = dataset.landmark_dim
    print(f"Clases ({num_classes}): {dataset.class_names}")
    print(f"Dimensión de entrada detectada: {input_dim}")
    print(f"Total muestras: {len(dataset)} | Train: {len(train_loader.dataset)} | "
          f"Val: {len(val_loader.dataset)}")

    model = LipReadingConformer(num_classes=num_classes, input_dim=input_dim).to(device)

    train_labels = [dataset._label_from_path(p) for p in train_loader.dataset.file_paths]
    class_counts = np.bincount(train_labels, minlength=num_classes).astype(np.float64)
    class_counts[class_counts == 0] = 1.0
    class_weights = class_counts.sum() / (num_classes * class_counts)
    class_weights_t = torch.tensor(class_weights, dtype=torch.float32, device=device)
    print("Pesos por clase (loss):")
    for name, w, c in zip(dataset.class_names, class_weights, class_counts):
        print(f"  {name:28s} {int(c):4d} muestras -- peso {w:.3f}")

    criterion = nn.CrossEntropyLoss(label_smoothing=label_smoothing, weight=class_weights_t)
    optimizer = optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
    scheduler = build_warmup_cosine_scheduler(optimizer, epochs, warmup_epochs)
    early_stopping = EarlyStopping(patience=early_stopping_patience, min_delta=0.001)

    os.makedirs(os.path.dirname(MODEL_PATH), exist_ok=True)
    best_val_loss = float("inf")

    history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}

    print("\nIniciando entrenamiento (Conformer)...")
    for epoch in range(epochs):
        train_loss, train_acc = run_epoch(model, train_loader, criterion, optimizer, device, train=True)
        val_loss, val_acc = run_epoch(model, val_loader, criterion, optimizer, device, train=False)
        scheduler.step()

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        history["train_acc"].append(train_acc)
        history["val_acc"].append(val_acc)

        lr_actual = optimizer.param_groups[0]["lr"]
        print(f"Época [{epoch+1}/{epochs}] | Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}% | "
              f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.2f}% | LR: {lr_actual:.6f}")

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            torch.save({
                "model_state_dict": model.state_dict(),
                "class_names": dataset.class_names,
                "max_frames": max_frames,
                "input_dim": input_dim,
                "val_loss": val_loss,
                "val_acc": val_acc,
            }, MODEL_PATH)
            print(f"  -> Nuevo mejor modelo guardado (Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%)")

        early_stopping(val_loss)
```

```
        if early_stopping.early_stop:
            print(f"  -> Early stopping activado en la época {epoch+1} "
                  f"(sin mejora en Val Loss por {early_stopping_patience} épocas).")
            break

    plot_training_curves(history["train_loss"], history["val_loss"],
                         history["train_acc"], history["val_acc"])

    print(f"\nListo. Mejor Val Loss: {best_val_loss:.4f} | Modelo en: {MODEL_PATH}")

    try:
        best_checkpoint = torch.load(MODEL_PATH, map_location=device)
        model.load_state_dict(best_checkpoint["model_state_dict"])
        evaluate_final_report(model, val_loader, dataset.class_names, device)
    except Exception as e:
        print(f"\n[AVISO] No se pudo generar el reporte de resultados: {e}")
        print("El modelo entrenado SÍ se guardó bien, esto solo afecta al reporte de métricas.")
```

8 main

scripts/train_audio_classifier.py:168-284

Entrena el modelo de audio.

```
168 def main():
169     clips_por_clase = find_labeled_clips()
170     if not clips_por_clase:
171         print(f"No encontré clips con audio en {SESSIONS_DIR}. Grabá primero con "
172               "grabar_video_continuo.py.")
173         return
174     print("Clases encontradas:")
175     for clase, wavs in clips_por_clase.items():
176         print(f"  {clase}: {len(wavs)} clips")
177 
178     device = "cuda" if torch.cuda.is_available() else "cpu"
179     todos_cacheados = all(
180         item.lower().endswith(".npy") for wavs in clips_por_clase.values() for item in wavs
181     )
182     if todos_cacheados:
183         print("\nTodos los embeddings ya están cacheados -- no hace falta cargar wav2vec2.")
184         model = feature_extractor = None
185     else:
186         print(f"\nCargando wav2vec2 para extraer embeddings (device={device})...")
187         model, feature_extractor, _vocab, device = load_model(device=device)
188 
189     print("\nExtrayendo embeddings (con caché, las corridas siguientes son instantáneas)...")
190     X, y, class_names = build_embeddings(clips_por_clase, model, feature_extractor, device)
191     print(f"\nTotal: {len(X)} clips, {len(class_names)} clases: {class_names}")
192 
193     train_idx, val_idx = stratified_split(y, VAL_FRACTION, SEED)
194     print(f"Train: {len(train_idx)} | Val: {len(val_idx)}")
195 
196     mean = X[train_idx].mean(axis=0, keepdims=True)
197     std = X[train_idx].std(axis=0, keepdims=True) + 1e-6
198 
199     def to_tensor(idx):
200         xs = (X[idx] - mean) / std
201         return torch.from_numpy(xs).float().to(device), torch.from_numpy(y[idx]).long().to(device)
202 
203     x_train, y_train = to_tensor(train_idx)
204     x_val, y_val = to_tensor(val_idx)
205 
206     head = AudioClassifierHead(EMBEDDING_DIM, HIDDEN_DIM, len(class_names), DROPOUT).to(device)
207     optimizer = torch.optim.Adam(head.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
208     criterion = nn.CrossEntropyLoss()
209 
210     history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}
211     best_val_acc = -1.0
212     best_state = None
213     early_stopping = EarlyStopping(patience=EARLY_STOPPING_PATIENCE, min_delta=0.001)
214     for epoch in range(1, EPOCHS + 1):
215         head.train()
216         optimizer.zero_grad()
217         logits = head(x_train)
218         loss = criterion(logits, y_train)
219         loss.backward()
220         optimizer.step()
221 
222         head.eval()
223         with torch.no_grad():
224             train_acc = (head(x_train).argmax(1) == y_train).float().mean().item()
225             val_logits = head(x_val)
226             val_loss = criterion(val_logits, y_val).item()
227             val_acc = (val_logits.argmax(1) == y_val).float().mean().item()
228 
229         history["train_loss"].append(loss.item())
230         history["val_loss"].append(val_loss)
231         history["train_acc"].append(train_acc * 100)
232         history["val_acc"].append(val_acc * 100)
233 
```

```
        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_state = {k: v.clone() for k, v in head.state_dict().items()}

        if epoch % 10 == 0 or epoch == 1:
            print(f"  epoch {epoch:3d} | loss {loss.item():.3f} | val_loss {val_loss:.3f} | "
                  f"train_acc {train_acc:.2f} | val_acc {val_acc:.2f}")

        early_stopping(val_loss)
        if early_stopping.early_stop:
            print(f"  -> Early stopping activado en la epoca {epoch} "
                  f"(sin mejora en val_loss por {EARLY_STOPPING_PATIENCE} epocas).")
            break

    plot_training_curves(history)

    print(f"\nMejor accuracy de validación: {best_val_acc:.2f}
({int(best_val_acc*len(val_idx))}/{len(val_idx)})")
    if len(val_idx) < 10:
        print("[AVISO] Set de validación muy chico (<10 clips) -- este número es orientativo, "
              "no una medición confiable todavía. Va a mejorar con más datos.")

    head.load_state_dict(best_state)
    head.eval()
    with torch.no_grad():
        val_pred = head(x_val).argmax(1).cpu().numpy()
    val_true = y[val_idx]
    print("\nMatriz de confusión (filas=real, columnas=predicho):")
    print("clases:", class_names)
    n = len(class_names)
    conf = np.zeros((n, n), dtype=int)
    for t, p in zip(val_true, val_pred):
        conf[t, p] += 1
    for i, row in enumerate(conf):
        print(f"  {class_names[i]:35s} {row.tolist()}")

    plot_confusion_matrix(conf, class_names, best_val_acc)

    os.makedirs(os.path.dirname(MODEL_OUT), exist_ok=True)
    torch.save(
        {
            "state_dict": best_state,
            "class_names": class_names,
            "embedding_dim": EMBEDDING_DIM,
            "hidden_dim": HIDDEN_DIM,
            "mean": mean,
            "std": std,
            "val_acc": best_val_acc,
        },
        MODEL_OUT,
    )
    print(f"\nGuardado en {MODEL_OUT}")
```

Pronunciación por fonema (GOP)

9 score_pronunciation

scripts/audio_pronunciation.py:296-361

Puntúa cada fonema de una palabra.

```python
def score_pronunciation(wav_path, palabra, model, feature_extractor, vocab, device):
    waveform = load_audio_16k(wav_path)

    inputs = feature_extractor(waveform.numpy(), sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")
    with torch.inference_mode():
        logits = model(inputs.input_values.to(device)).logits
    log_probs = torch.log_softmax(logits, dim=-1)

    id_to_phoneme = {v: k for k, v in vocab.items()}
    blank_id = vocab["<pad>"]

    pred_ids = torch.argmax(log_probs, dim=-1)[0].tolist()
    reconocido_libre = _greedy_ctc_decode(pred_ids, id_to_phoneme, blank_id)

    fonemas_esperados = texto_a_fonemas(palabra)
    target_ids, fonemas_usados = _phoneme_to_token_ids(fonemas_esperados, vocab)
    if not target_ids:
        raise ValueError(f"No se pudo mapear ningún fonema de '{palabra}' al vocabulario del modelo.")

    targets = torch.tensor([target_ids], dtype=torch.int32, device=device)

    aligned_labels, aligned_scores = F.forced_align(
        log_probs, targets, blank=blank_id,
    )
    aligned_labels = aligned_labels[0].tolist()
    aligned_scores = aligned_scores[0].tolist()

    scores_por_fonema = []
    idx_target = 0
    frame_scores_actual = []
    for label, score in zip(aligned_labels, aligned_scores):
        if label == blank_id:
            continue
        if idx_target < len(target_ids) and label == target_ids[idx_target]:
            frame_scores_actual.append(score)
        elif frame_scores_actual:
            scores_por_fonema.append(float(np.mean(frame_scores_actual)))
            frame_scores_actual = [score]
            idx_target += 1
        else:
            frame_scores_actual = [score]
    if frame_scores_actual:
        scores_por_fonema.append(float(np.mean(frame_scores_actual)))

    while len(scores_por_fonema) < len(fonemas_usados):
        scores_por_fonema.append(None)

    sospechosos = [
        i for i, s in enumerate(scores_por_fonema)
        if s is not None and s < GOP_SUSPICIOUS_THRESHOLD
    ]

    distancia = _distancia_edicion(fonemas_usados, reconocido_libre)
    normalizada = distancia / max(len(fonemas_usados), len(reconocido_libre), 1)
    palabra_reconocida = normalizada <= PALABRA_MISMATCH_THRESHOLD

    return {
        "fonemas": fonemas_usados,
        "scores": scores_por_fonema,
        "sospechosos": sospechosos,
        "reconocido_libre": " ".join(reconocido_libre),
        "sonido_reconocido": fonemas_a_texto_aproximado(reconocido_libre),
        "distancia_normalizada": normalizada,
```

```
359         "palabra_reconocida": palabra_reconocida,
360         "tipo_error_audio": diagnose_tipo_error(fonemas_usados, reconocido_libre),
361     }
```

10 score_word_in_sentence

scripts/audio_pronunciation.py:385-498

Hace lo mismo con una palabra dentro de una frase.

```python
def score_word_in_sentence(wav_path, frase, palabra_objetivo, model, feature_extractor, vocab, device):
    waveform = load_audio_16k(wav_path)

    inputs = feature_extractor(waveform.numpy(), sampling_rate=TARGET_SAMPLE_RATE, return_tensors="pt")
    with torch.inference_mode():
        logits = model(inputs.input_values.to(device)).logits
    log_probs = torch.log_softmax(logits, dim=-1)

    id_to_phoneme = {v: k for k, v in vocab.items()}
    blank_id = vocab["<pad>"]

    pred_ids = torch.argmax(log_probs, dim=-1)[0].tolist()
    reconocido_libre = _greedy_ctc_decode(pred_ids, id_to_phoneme, blank_id)

    fonemas_frase = texto_a_fonemas(frase)
    fonemas_palabra = texto_a_fonemas(palabra_objetivo)
    target_ids, fonemas_frase_usados = _phoneme_to_token_ids(fonemas_frase, vocab)
    if not target_ids:
        raise ValueError(f"No se pudo mapear ningún fonema de la frase '{frase}' al vocabulario del modelo.")

    targets = torch.tensor([target_ids], dtype=torch.int32, device=device)
    aligned_labels, aligned_scores = F.forced_align(log_probs, targets, blank=blank_id)
    aligned_labels = aligned_labels[0].tolist()
    aligned_scores = aligned_scores[0].tolist()

    scores_por_fonema = []
    frame_ranges = []
    idx_target = 0
    frame_scores_actual = []
    frame_indices_actual = []
    for frame_idx, (label, score) in enumerate(zip(aligned_labels, aligned_scores)):
        if label == blank_id:
            continue
        if idx_target < len(target_ids) and label == target_ids[idx_target]:
            frame_scores_actual.append(score)
            frame_indices_actual.append(frame_idx)
        elif frame_scores_actual:
            scores_por_fonema.append(float(np.mean(frame_scores_actual)))
            frame_ranges.append((frame_indices_actual[0], frame_indices_actual[-1]))
            frame_scores_actual = [score]
            frame_indices_actual = [frame_idx]
            idx_target += 1
        else:
            frame_scores_actual = [score]
            frame_indices_actual = [frame_idx]
    if frame_scores_actual:
        scores_por_fonema.append(float(np.mean(frame_scores_actual)))
        frame_ranges.append((frame_indices_actual[0], frame_indices_actual[-1]))
    while len(scores_por_fonema) < len(fonemas_frase_usados):
        scores_por_fonema.append(None)
        frame_ranges.append(None)

    inicio, fin = _localizar_subsecuencia(fonemas_frase_usados, fonemas_palabra)
    fonemas_usados = fonemas_frase_usados[inicio:fin]
    scores_recortados = scores_por_fonema[inicio:fin]

    rangos_validos = [r for r in frame_ranges[inicio:fin] if r is not None]
    audio_recortado = None
    if rangos_validos:
        total_frames = log_probs.shape[1]
        total_muestras = waveform.shape[0]
        muestras_por_frame = total_muestras / max(total_frames, 1)
        frame_inicio = min(r[0] for r in rangos_validos)
        frame_fin = max(r[1] for r in rangos_validos)
        colchon_muestras = int(0.15 * TARGET_SAMPLE_RATE)
        muestra_inicio = max(0, int(frame_inicio * muestras_por_frame) - colchon_muestras)
```

```
        muestra_fin = min(total_muestras, int((frame_fin + 1) * muestras_por_frame) + colchon_muestras)
        if muestra_fin > muestra_inicio:
            audio_recortado = waveform[muestra_inicio:muestra_fin]

    sospechosos = [
        i for i, s in enumerate(scores_recortados)
        if s is not None and s < GOP_SUSPICIOUS_THRESHOLD
    ]

    scores_fuera_de_la_palabra = scores_por_fonema[:inicio] + scores_por_fonema[fin:]
    sospechosos_fuera = [
        s for s in scores_fuera_de_la_palabra
        if s is not None and s < GOP_SUSPICIOUS_THRESHOLD
    ]
    resto_de_la_frase_ok = len(sospechosos_fuera) < 2

    lib_inicio, lib_fin = _localizar_subsecuencia(reconocido_libre, fonemas_palabra)
    ventana_reconocida = reconocido_libre[lib_inicio:lib_fin]
    distancia = _distancia_edicion(ventana_reconocida, fonemas_palabra)
    normalizada = distancia / max(len(fonemas_palabra), len(ventana_reconocida), 1)
    palabra_reconocida = normalizada <= FRASE_PALABRA_MISMATCH_THRESHOLD

    frase_completa_dicha = (
        len(reconocido_libre) >= FRASE_COMPLETA_LARGO_MINIMO * len(fonemas_frase_usados)
    )

    audio_recortado_path = None
    if audio_recortado is not None:
        import tempfile
        with tempfile.NamedTemporaryFile(delete=False, suffix=".wav") as tmp:
            audio_recortado_path = tmp.name
        torchaudio.save(audio_recortado_path, audio_recortado.unsqueeze(0), TARGET_SAMPLE_RATE)

    return {
        "frase": frase,
        "palabra_objetivo": palabra_objetivo,
        "fonemas": fonemas_usados,
        "scores": scores_recortados,
        "sospechosos": sospechosos,
        "reconocido_libre": " ".join(reconocido_libre),
        "sonido_reconocido": fonemas_a_texto_aproximado(ventana_reconocida),
        "distancia_normalizada": normalizada,
        "palabra_reconocida": palabra_reconocida,
        "frase_completa_dicha": frase_completa_dicha,
        "resto_de_la_frase_ok": resto_de_la_frase_ok,
        "tipo_error_audio": diagnose_tipo_error(fonemas_palabra, reconocido_libre),
        "audio_recortado_path": audio_recortado_path,
    }
```